	JSPM University Pune **Faculty of Science and Technology** **School of Computational Sciences** **SY B. Tech. Lateral (CSE&IT)**
Experiment No: 6	**Date: / /**
Aim: Write an ALP to subtract the bytes in Internal RAM 34h & 35h and store the result in register R5 (LSB) & R6 (MSB).	

Aim of the Practical

1. Understand **subtraction operations** in **8051 Assembly Language**.
2. Learn how to use **Indirect Addressing Mode** for memory operations.
3. Implement **borrow handling** for **16-bit subtraction**.
4. Perform arithmetic operations using **internal RAM of the 8051 microcontroller**.
5. Simulate and test the program using **Keil uVision** or **real hardware**.

Equipments Required:

- 8051 Microcontroller Kit (or Keil uVision Simulator)
- Personal Computer with Keil Software
- Power Supply
- USB to Serial Cable (if using hardware)

Theory

The **8051 microcontroller** has **128 bytes of internal RAM (00H – 7FH)**. **Indirect Addressing Mode** is used to access memory locations dynamically using **registers as pointers** (R0 or R1).

In subtraction, the **Carry (CY) flag** is used to handle **borrow conditions**. The **SUBB (Subtract with Borrow) instruction** is used to perform **subtraction** and account for any previous borrow.

Working Mechanism

1. Load the **source addresses (34H and 35H)** into **R0**.
2. Fetch the first number (Minuend) from **@R0 (34H)** into the **A register**.
3. Increment R0 to point to **35H** and fetch the second number (Subtrahend).
4. Perform **subtraction** and store the **LSB in R5**.
5. Store the **borrow (if any) in R6 (MSB)**.

Algorithm :-

Code:-

Program	Comments

Conclusion

	JSPM University Pune **Faculty of Science and Technology** **School of Computational Sciences** **SY B. Tech. Lateral (CSE&IT)**
Experiment No: 7	**Date: / /**
Aim: Write an assembly language program to multiply two 8-bit numbers stored at 30h and 31h and store16- bit results in 32h and 33h of Internal RAM.	

Aim of the Practical

1. Understand **multiplication operations** in **8051 Assembly Language**.
2. Learn how to use **registers (A and B) for arithmetic operations**.
3. Implement **handling of 16-bit results using RAM storage**.
4. Perform arithmetic operations using **internal RAM of the 8051 microcontroller**.
5. Simulate and test the program using **Keil uVision** or **real hardware**.

Equipments Required:

- 8051 Microcontroller Kit (or Keil uVision Simulator)
- Personal Computer with Keil Software
- Power Supply
- USB to Serial Cable (if using hardware)

Theory

The **8051 microcontroller** supports **8-bit multiplication** using the `MUL` instruction. The result of multiplying two **8-bit numbers** can be a **16-bit value** stored in **two registers**:

- **Lower 8-bits (LSB)** in **Accumulator (A)**
- **Higher 8-bits (MSB)** in **Register B (B)**

To perform the multiplication:

1. Load the first number from **30H** into the **Accumulator (A)**.
2. Load the second number from **31H** into **Register B (B)**.
3. Execute the `MUL AB` instruction.
4. Store the **LSB (A) at 32H** and **MSB (B) at 33H** in RAM.

Algorithm :-

Code :-

Program	Comments

Conclusion

	JSPM University Pune **Faculty of Science and Technology** **School of Computational Sciences** **SY B. Tech. Lateral (CSE&IT)**
Experiment No: 8	**Date: / /**
Aim: Write an assembly language program to perform a division operation on the 8-bit number by an 8-bit number.	

Aim of the Practical

1. Understand **division operations** in **8051 Assembly Language**.
2. Learn how to use **registers (A and B) for arithmetic operations**.
3. Implement **handling of quotient and remainder using RAM storage**.
4. Perform arithmetic operations using **internal RAM of the 8051 microcontroller**.
5. Simulate and test the program using **Keil uVision** or **real hardware**.

Equipments Required:

- 8051 Microcontroller Kit (or Keil uVision Simulator)
- Personal Computer with Keil Software
- Power Supply
- USB to Serial Cable (if using hardware)

Theory

The **8051 microcontroller** provides the `DIV` instruction, which is used to **divide two 8-bit numbers**.

- The **dividend** is stored in the **Accumulator (A)**.
- The **divisor** is stored in the **B register**.
- The `DIV AB` instruction performs **A ÷ B**, storing:
  - **Quotient in A**
  - **Remainder in B**

If **B = 00H**, a **divide-by-zero error** occurs, setting the **overflow flag (OV = 1)**.

Algorithm :-

Code :-

Program	Comments

Conclusion

	JSPM University Pune **Faculty of Science and Technology** **School of Computational Sciences** **SY B. Tech. Lateral (CSE&IT)**
Experiment No: 9	**Date: / /**
Aim: Write an assembly language program to separate positive and negative in a given array.	

Aim of the Practical

1. Understand signed number representation in 8051.
2. Learn bitwise operations for checking positive and negative numbers.
3. Use indirect addressing mode for array processing.
4. Simulate and test the program using Keil uVision.

Equipments Required:

- 8051 Microcontroller Kit (or Keil uVision Simulator)
- Personal Computer with Keil Software
- Power Supply
- USB to Serial Cable (if using hardware)

Theory

In the **8051 microcontroller**, signed numbers are represented in **8-bit 2's complement format**.

- **Positive numbers**: The **MSB (Most Significant Bit) is 0** (Range: **00H to 7FH**).
- **Negative numbers**: The **MSB is 1** (Range: **80H to FFH**, in **2's complement form**).

Working Mechanism

1. Read **each element** of the given array.
2. Check the **MSB (7th bit, ACC.7)** of the number:
   - If **MSB = 0**, store it in the **positive array**.
   - If **MSB = 1**, store it in the **negative array**.
3. Repeat until all elements are processed.

Algorithm

1. **Initialize registers**:
   - Load the **source array** starting address.
   - Load **destination addresses** for positive and negative numbers.
   - Set a **counter** for the number of elements.

2. **Loop through each element**:
    - Load a number from the array.
    - Check the **MSB (ACC.7)** to determine whether the number is **positive or negative**.
    - Store the number in the appropriate memory location.
3. **Continue processing until all numbers are checked**.

Observations

1. **Numbers with MSB = 0** were successfully stored in the **positive array**.
2. **Numbers with MSB = 1** were correctly identified as **negative numbers** and stored in the **negative array**.
3. The program correctly separated **positive and negative numbers** using bitwise checking.

Code :-

Program	Comments

Conclusion

	JSPM University Pune **Faculty of Science and Technology** **School of Computational Sciences** **SY B. Tech. Lateral (CSE&IT)**
Experiment No: 10	**Date: / /**
Aim: Write an ALP to separate even or odd elements in a given array.	

Aim of the Practical

1. To develop an Assembly Language Program (ALP) using Keil software for separating even and odd numbers from a given array.
2. To implement and execute the program on an 8051 microcontroller-based simulator.
3. To analyze the results by verifying the separated even and odd numbers in memory.

Equipments Required:

- 8051 Microcontroller Kit (or Keil uVision Simulator)
- Personal Computer with Keil Software
- Power Supply
- USB to Serial Cable (if using hardware)

Theory :-

Assembly Language Programming (ALP) allows direct hardware control and efficient execution, making it suitable for low-level operations like array manipulation. The given task is to separate even and odd elements from an array.

Algorithm :-

Code :-

Program	Comments

Conclusion:-